AI JOB GUIDE — INGESTION AUDIT

Lebanon Job Coverage Audit

A read-only assessment of how much real Lebanon job coverage the ingestion pipeline currently produces, where it comes from, and what's actually worth doing next — against the live local database, not assumptions.

Scope: local Supabase, read-only **Baseline:** 857 total jobs **Date:** 2026-10-25

> **Verdict:** Lebanon coverage is real, not negligible — 72 jobs tagged Lebanon (46 active), fresh, well-located, 100% with a usable apply target. The gap isn't "no coverage," it's *concentration*: one source (Bayt) carries two-thirds of it, a second proven source (LinkedIn) sits disabled behind a human decision, and a few specific, already-diagnosed engineering gaps (not new-source hunting) are the highest-leverage next moves.

72	46	~41	3	100%
jobs tagged Lebanon	currently active	active after de-duplication	sources contributing any Lebanon jobs	active Lebanon jobs with a real apply URL

1 — 3 · THE NUMBERS

What's actually in the database

Every row counted is either tagged `country_code = 'LB'` or carries "Lebanon"/"Beirut" in its location text — the two methods agreed exactly (0 normalization mismatches), so the location-tagging pipeline itself is sound for the jobs it does see.

Lebanon jobs by source

SOURCE	LEBANON JOBS	SOURCE TOTAL (DB-WIDE)	LEBANON SHARE
Bayt	49	49	100%
LinkedIn	13	47	28%
Oracle HCM (AUBMC only)	10	50	20%
Greenhouse / Lever / Workable / Ashby / GulfTalent / Indeed / RemoteOK / Jobicy / Arbeitnow	0	513	—

Lebanon jobs by status & freshness

STATUS	COUNT	FRESHNESS (ACTIVE ONLY, BY LAST_SEEN_AT)
`active`	46	all 46 are 8–30 days old; none ≤7d, none >30d
`unavailable`	26	correctly stale-closed, not deleted

Location-field completeness (Lebanon jobs)

COMPLETENESS	COUNT	SHARE
City + country	39	54%
Country only	33	46%
Neither	0	0%

FINDING

5 of 6 "possible duplicates" are the same real job, twice

A title+company collision scan found 6 pairs. Five are the exact same LinkedIn posting under two different `external_id` conventions; one is a coincidental collision between two different Bayt postings.

`confirmed`

Root cause: the real adapter (`providers/linkedin.ts`) derives `externalId: String(raw.id)` — a bare numeric LinkedIn job id. Five rows instead carry a `phase1-real-pilot-2026-10-17074910-<id>` prefix for the *identical* job (same `application_url`). Since the dedup unique index is `(dedup_scope, external_id)`, two different id strings for one real job both pass as "new," producing a true duplicate row.

```
4477362945 -> "phase1-real-pilot-2026-10-17074910-4477362945" (active) + "4477362945" (active)
4241205848 / 4236879065 / 4467352986 -> same pattern
```

These are leftover rows from an earlier pilot script's own test-id naming — never cleaned up — not a defect in the current adapter. Net effect: the real distinct active Lebanon count is closer to **~41**, not 46.

`not a duplicate`

Two Bayt rows titled "Administrative Assistant" at "Confidential Company" — different `external_id` hashes, different listing URLs. Both employers anonymized identically; this is a coincidental collision, not a data defect.

4 — 5 · SOURCE CONFIGURATION

Why only 3 sources show up for Lebanon

588 `company_sources` rows exist in the registry; 130 target Lebanon (84 verified, 46 awaiting manual review). Of the 84 verified, only a handful carry an ATS type this pipeline has an adapter for.

Verified Lebanon company_sources, by ATS type this pipeline can actually automate

ATS TYPE	VERIFIED COS.	CURRENTLY INGESTING LEBANON JOBS?
Oracle Cloud Recruiting	1	yes — AUBMC, 10 jobs
Oracle HCM Cloud (candidate-portal variant)	1	unconfirmed — Norwegian Refugee Council
Lever	2	no — Wahed, Welo Data: 0 Lebanon-tagged jobs
Workable	2	no jobs currently tagged Lebanon
Greenhouse	1	no jobs currently tagged Lebanon
Workday	1	no — architecture gap, see below
SAP SuccessFactors / Zenats / Teamtailor / BambooHR / JobTeaser / Freshteam / Phenom People / Intervieweb	11	no adapter exists for any of these today
unknown / custom / none / not_applicable	112	no identified ATS — career-page extraction candidates

dormant, already proven

LinkedIn (`providerConfig.ts`) is live-verified twice against real Lebanon data, has Lebanon in its `markets`, and is fully implemented — but still sits at `enabled:false`. The code's own comment is explicit: the only remaining gate is a deliberate human approval step, not more engineering.

real, scoped engineering gap

Workday / Murex — a real, named Lebanon employer, confirmed reachable with no auth — is blocked because Workday's list endpoint carries no job description field at all, and every real job is honestly rejected by `missing_description` until a second per-job detail fetch is added. This is the one concrete "fix code, not find a new source" opportunity identified.

measurement nuance

Lever/Workable/Greenhouse companies verified for Lebanon show *zero* Lebanon-tagged jobs not necessarily because they have none, but because remote-first employers (Wahed, Welo Data) post with location text like "Remote" rather than "Lebanon" — invisible to a country-code count even though the registry correctly flags them as Lebanon-relevant. Raw counts likely *undercount* true Lebanon-accessible opportunities.

`confirmed closed, not a gap`

GulfTalent and **Indeed** (both Apify-backed) were live-verified this project to have no Lebanon option in their own country enums — a real, zero-cost finding, not an oversight. **Adzuna**'s documented country list excludes Lebanon outright. None of these three are closeable Lebanon gaps regardless of future credentials.

6 — 7 · WHAT TO DO NEXT

Prioritized, cost-conscious plan

Ordered by leverage: fixes and decisions on sources already built come first; anything requiring a brand-new paid provider comes last, and only if the earlier steps don't close the gap.

1 **Clean up the leftover LinkedIn pilot duplicates**

Delete exactly the 5 confirmed `phase1-real-pilot-*`-prefixed rows (same real job as a bare-id row already present). Scoped, exact-id delete — not a cleanup sweep.

`$0` `data hygiene`

2 **Fix LinkedIn's external_id convention before any wider run**

Confirm every ingestion path (pilot scripts included) uses the adapter's bare `String(raw.id)` convention, so future runs can't reproduce the same duplicate pattern at larger scale.

`$0` `small scoped fix`

3 **Decide on LinkedIn Lebanon enablement**

Already live-verified twice, already in-scope for Lebanon, already implemented. The only remaining step is the explicit human approval call this project's own guardrails require — not new work.

`$0 (free tier observed)` `decision, not build`

4 **Re-point career-page extraction at the Lebanon "unknown ATS" cohort**

112 verified/review Lebanon companies have no identified ATS. The extraction pipeline (Tier B) is already built and live-tested — it was just never run against this specific cohort. Zero new code.

`$0` `reuse existing pipeline`

5 **Close the Workday detail-fetch gap**

Add the second per-job request Workday's architecture requires. Unlocks Murex today and any future Workday-ATS company found, in any market.

`$0` `scoped engineering`

6 **Work the 46 needs_manual_review Lebanon registry rows**

Registry curation, not engineering — identifying real ATS types for these could surface more automatable candidates without touching a new source.

`$0` `registry curation`

7 **Only then: evaluate a genuinely new paid source for Lebanon**

No uncovered Lebanon-specific gap currently justifies new spend — Bayt already covers Lebanon well, and GulfTalent/Indeed are confirmed structurally Gulf-only. If a candidate ever emerges, benchmark it the same way this project already proved out: a free schema/country-enum check first, then the smallest possible bounded paid sample under an explicit platform-level cost ceiling (never an input-level "maxItems" alone — this project's own $1.47-for-10-requested incident is why), checked for Lebanon signal and external-id stability, before any `enabled:true` — and only with explicit sign-off, never automatically.

`last resort` `full guardrail discipline required`

Read-only assessment against the local Supabase instance, 857 baseline jobs, zero writes performed. No sources enabled, no workflows executed, no provider calls made.